SC4001 Programming Assignment: Part A Report

Name: Avneesh Sridhar Matric No.: U2323593F

Generative AI Declaration: I used Claude to help debug the code and for aid in ideation and the coding for some sections. In the report, I used the same for polishing it and creating tables out of the output data. I also used it for labelling and formatting of the output images and tables, and to reference them in the text.

A1. Baseline, grid search and final model

The data was split by year into train (2017-2020), validation (2021) and test (2022). All the models were trained with Adam (learning rate 0.01, batch size 512) using MSE loss on the standardised price. Training ran for up to 20 epochs, with early stopping on the validation RMSE, using a patience of 4 epochs and a minimum improvement of 1e-4.

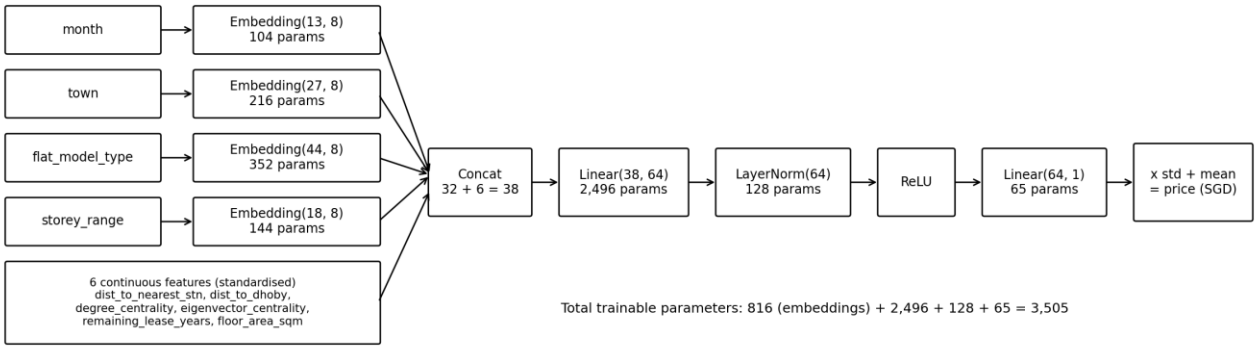


Figure 1. Baseline model. It has 3,505 trainable parameters.

For the grid search, Ray Tune was used to train all 25 combinations of hidden width {16, 32, 64, 128, 256} and embedding dimension {2, 4, 8, 12, 16}, using the same embedding size for all four categorical features.

Table 1. Best validation RMSE (SGD thousands) for each configuration.

emb \ hidden	**16**	**32**	**64**	**128**	**256**
2	65.9	67.5	62.8	64.9	64.8
4	63.9	62.8	63.7	**60.1**	62.2
8	66.5	67.4	65.0	64.5	62.3
12	63.4	62.9	64.2	62.3	63.5
16	63.6	61.3	62.7	65.6	61.8

The selected configuration is hidden width 128 and embedding dimension 4, with a best validation RMSE of $60,106 at epoch 6. It was then retrained on 2017-2021 for 6 epochs, its best epoch, since there is no validation set left for early stopping. On 2022 it gets a test RMSE of $104,827 and a test R^2 of 0.621.

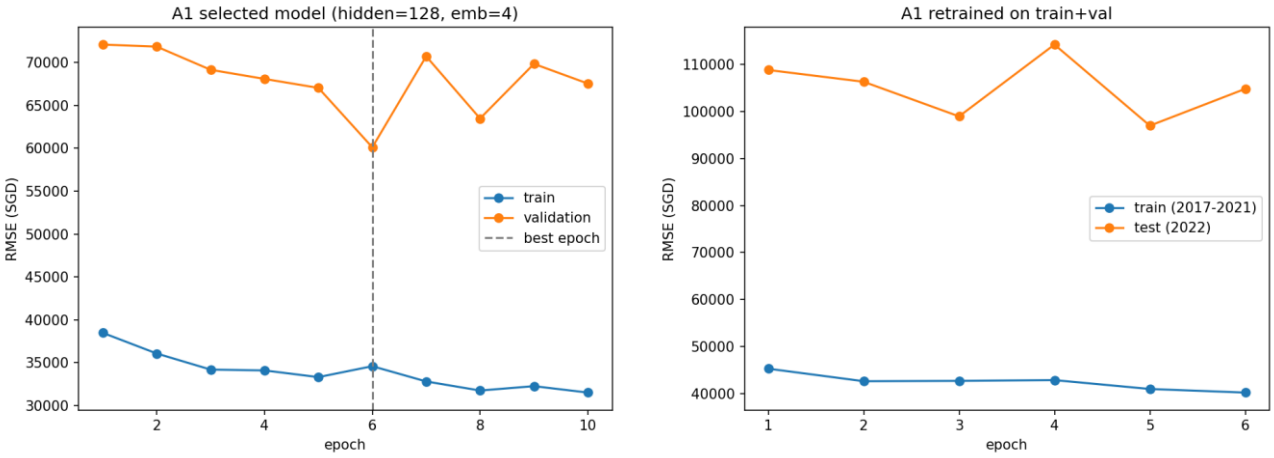


Figure 2. Left: training and validation RMSE of the selected configuration. Right: training and test RMSE after retraining on 2017-2021.

Most of the configurations in Table 1 ended up close to each other, between $60.1k and $67.5k, and there isn't a clear pattern where wider layers or bigger embeddings do better. This makes sense since the categorical features only have 13-44 categories each, so a small embedding is already enough to represent them. Hidden width 128 with embedding size 4 was picked because it had the lowest best validation RMSE, and it is still a fairly small model with 3,737 parameters.

In the learning curves, training RMSE falls steadily from about $38k to $31k, while validation RMSE fluctuates between $60k and $72k, reaching its minimum at epoch 6. There was no improvement in the next 4 epochs and training stopped at epoch 10.

There is a big gap between the training RMSE of $35k and the validation RMSE of $60k, which could look like overfitting. But the validation RMSE doesn't keep rising as training RMSE falls, so this isn't really overfitting. The gap mostly comes from house prices going up over time: the average price was about $440k in 2017-2020, $511k in 2021 and $550k in 2022. The model has no year input, so it keeps predicting at the older price level, and on 2022 it under-predicts by $93,869 on average. So the model doesn't clearly underfit or overfit; most of the test error comes from the price increase between years.

A2. Ablation experiments

For the ablations, the selected model with hidden width 128 and embedding size 4 was tried with two changes: (a) removing the categorical inputs completely, so the first layer only gets the 6 continuous features, and (b) swapping the ReLU in the hidden layer for a Sigmoid. The baseline and both ablations were trained on 2017-2021 for 6 epochs with seeds 10, 13, 42, 33 and 104, and each run was tested once on 2022.

Table 2. Test results over 5 seeds (sample standard deviation).

Model	Params	Test RMSE (mean ± s.d.)	Test R^2 (mean ± s.d.)
Baseline (A1)	3,737	$102,331 ± 1,462	0.639 ± 0.010
(a) No categorical	1,281	$110,191 ± 7,464	0.580 ± 0.056
(b) Sigmoid	3,737	$104,957 ± 6,747	0.619 ± 0.049

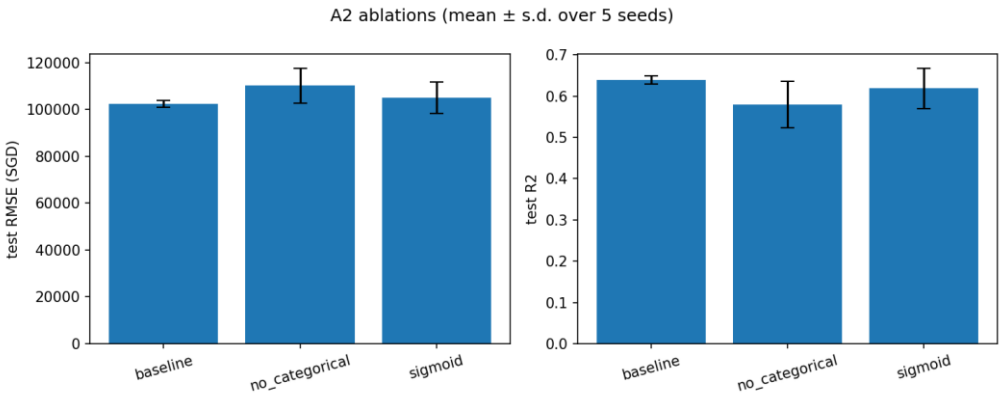


Figure 3a. Test RMSE and R^2 (mean ± s.d. over 5 seeds).

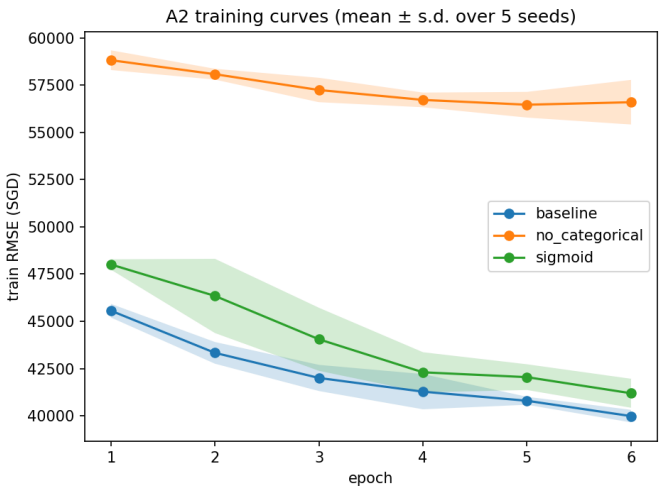


Figure 3b. Training RMSE per epoch (mean ± s.d. over 5 seeds).

(a) Removing the categorical features makes the model worse. The RMSE goes up by about $7.9k and R^2 drops from 0.639 to 0.580. In Figure 3b the training RMSE flattens out at around $56.5k, compared with $40k for the baseline, so the model is underfitting. Without town, flat type and storey, the six continuous features just don't carry enough information. The results also vary a lot more between seeds, with a standard deviation of $7.5k compared with $1.5k, so the model depends much more on how it is initialised. One seed, 104, still did better than the baseline, which suggests the test RMSE is also affected by the 2022 price increase, not just by how well the model fits.

(b) Using Sigmoid instead of ReLU is slightly worse and less stable. The average RMSE is $2.6k higher, but this is within one standard deviation, so with only 5 seeds it isn't a clear difference. The training curve starts higher, at $48.0k compared with $45.5k at epoch 1, and improves more slowly. This is expected, since sigmoid saturates and its gradient is never more than 0.25. The LayerNorm before the activation probably helps here, because it keeps the inputs close to zero, where sigmoid behaves almost linearly. The Sigmoid results also vary more across seeds, with a standard deviation about 4.6 times larger than the baseline.

Overall, the categorical embeddings are the most important part of the model, and ReLU gives slightly better and more consistent results than Sigmoid.

A3. Wide-and-deep architecture

The wide-and-deep model keeps the same MLP as before but adds a single linear layer, Linear(22, 1), that takes the same concatenated input. The two outputs are combined as output = MLP + λ × linear. This only adds 23 parameters, giving 3,760 in total compared with 3,737 for the baseline. λ was chosen with the A1 settings kept fixed, training on 2017-2020 with early stopping on 2021 and seed 42.

Table 3. Best validation RMSE for each λ.

λ	Best val RMSE	Best epoch
0	$64,995	5
0.25	$64,112	5
0.5	$63,514	5
1	$60,663	5
2	**$59,908**	**5**

Table 4. Test results over the five seeds.

Model	Params	Test RMSE (mean ± s.d.)	Test R^2 (mean ± s.d.)
Baseline (A1)	3,737	$102,331 ± 1,462	0.639 ± 0.010
Wide-and-deep (λ = 2)	3,760	$102,362 ± 2,238	0.638 ± 0.016

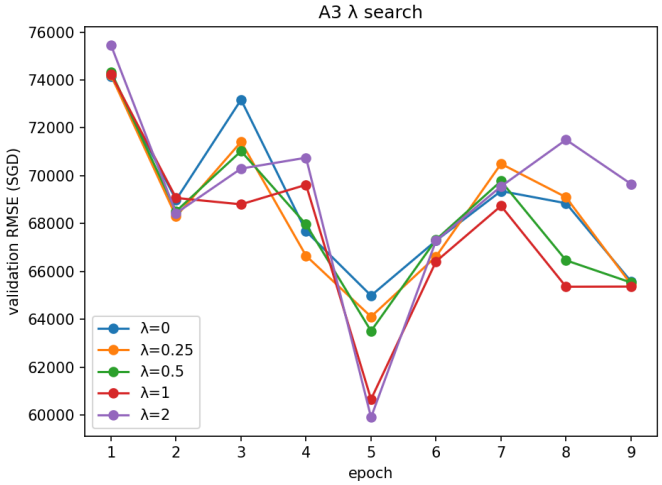


Figure 4a. Validation RMSE per epoch for each λ.

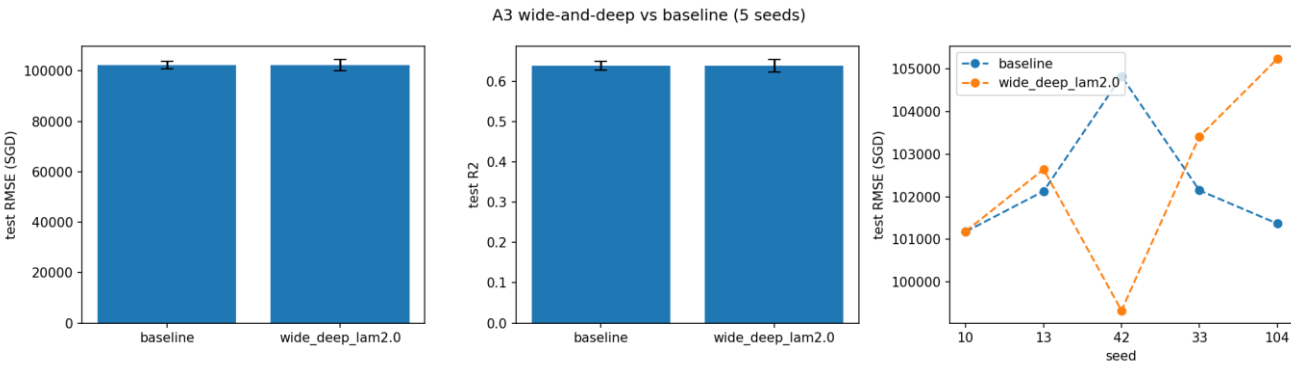


Figure 4b. Test RMSE and R^2 (mean ± s.d.) and test RMSE per seed.

λ = 2 was selected because it gave the lowest validation RMSE of $59,908, and the validation RMSE got better steadily as λ increased.

The linear branch gives the model a direct path from the inputs to the output. For optimisation, this means the gradients reach the embeddings and features more easily. The linear part can also pick up simple, near-linear effects like floor area and remaining lease quickly, which leaves the MLP to focus on the non-linear patterns and should help early in training. For generalisation, a linear term tends to behave more predictably than ReLU units when the inputs move outside the training range. λ controls how much the linear branch contributes.

The results only partly support this. On validation, a larger λ was better, going from $65.0k at λ = 0 to $59.9k at λ = 2. On the test set, though, wide-and-deep made no real difference: $102,362 ± 2,238 compared with $102,331 ± 1,462 for the baseline. It was only better on one of the five seeds, seed 42, and its results varied more. One reason is that a 128-unit ReLU MLP can already learn linear relationships, so the extra branch doesn't add much. Also, neither model can fix the 2022 price increase discussed in A1.

There are a few limitations in this comparison. First, λ was chosen using only one seed, and all five validation curves in Figure 4a have the same sharp dip at epoch 5. Since every run sees the batches in the same order, most of the curve shape comes from the data order rather than from λ, so the ranking isn't very reliable. Second, adding the extra layer changes the random initialisation of the MLP. λ = 0 should behave exactly like the baseline, but it got $65.0k compared with $60.1k in A1, so the comparison mixes the effect of the architecture with the effect of initialisation. Finally, five seeds is a small number, so small differences can't be trusted much.

A4. Feature importance (Captum FeatureAblation)

Both the final baseline from A1 and the wide-and-deep model (λ = 2) were trained with seed 42 on 2017-2021 for 6 epochs. Captum's FeatureAblation was then run on every 2022 test sample, using the predicted price in SGD. Each of the 10 original features was replaced one at a time with a baseline value: the most common category in the training set for categorical features, and the training mean for continuous features. For a categorical feature, its whole embedding input is replaced together. The importance score for each feature is the average absolute attribution across all test samples.

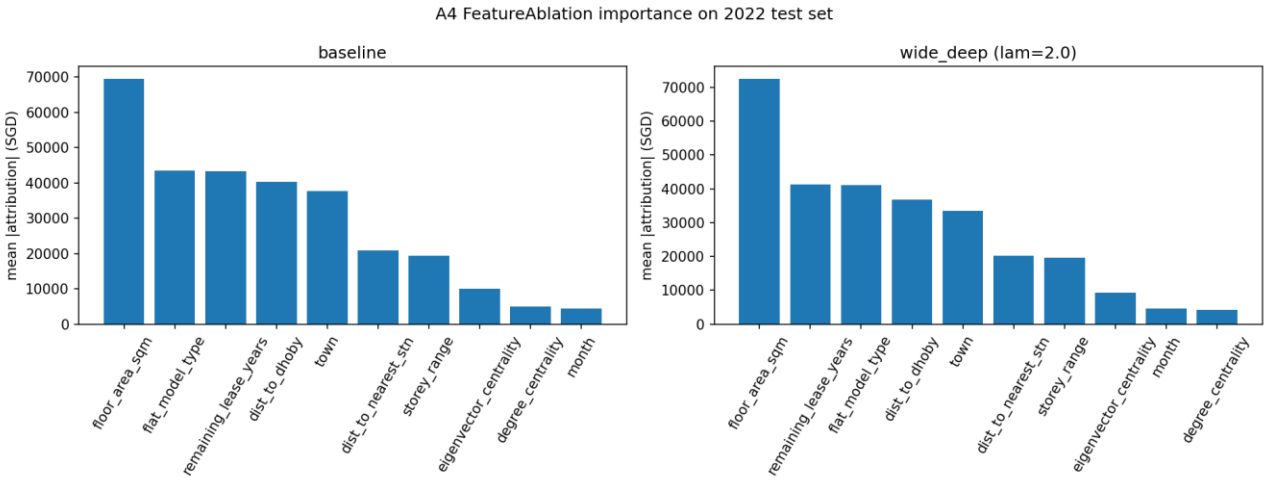


Figure 5. Global feature importance (mean |attribution|, SGD), ordered from highest to lowest.

Table 5. Three most important features.

Rank	Baseline	Wide-and-deep (λ = 2)
1	floor_area_sqm ($69.4k)	floor_area_sqm ($72.5k)
2	flat_model_type ($43.5k)	remaining_lease_years ($41.3k)
3	remaining_lease_years ($43.3k)	flat_model_type ($41.2k)

The two models give very similar results. Floor area is clearly the most important feature in both. Flat type and remaining lease are almost tied for 2nd and 3rd place, and they swap places between the two models. The location features, dist_to_dhoby and town, come next, while month and degree_centrality matter the least, at under $5k each. This fits the data description: bigger flats, longer leases and more central locations cost more, while the month of sale barely matters. The wide-and-deep model puts a bit more weight on floor area, about $3.1k more, and a bit less on location, with town down by $4.2k and dist_to_dhoby by $3.4k. This makes sense, since its linear branch can directly capture the roughly linear relationship between flat size and price.

One limitation is that several features carry overlapping information. Town, dist_to_dhoby, dist_to_nearest_stn and the two centrality measures all describe location in some way. When one of them is ablated, the others still hold most of that information, so each one gets less credit than location deserves as a whole. Ablating one feature at a time can also create unrealistic flats, such as a central distance paired with a far-away town, so the scores also depend on the baseline values chosen.